

hijacking may be possible.

X-Content-Type-Options header missing (7). The script and stylesheet elements will reject responses with incorrect MIME types if the server sends the response header 'X-Content-Type-Options: nosniff'. This is a security feature which prevents MIME-type confusion based attacks.

X-Frame-Options header not set (7). In this case, X-Frame-Options header is not included in the HTTP response to protect against Clickjacking attacks. The X-Frame-Options HTTP response header can be used to indicate whether or not a browser should be allowed to render a page in a <frame> or <iframe>. Sites can use this to avoid clickjacking attacks by ensuring that their content is not embedded into other sites.

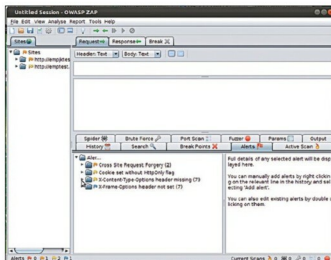


Fig. 4: OWASP ZAP test results

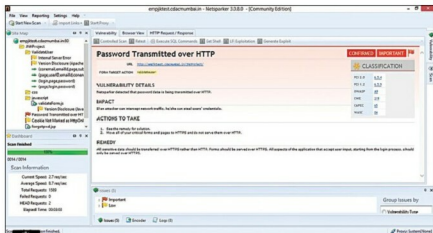


Fig. 5: Netsparker Community Edition test results

Testing results with Netsparker

Using the scanner feature of Netsparker Community Edition, we discovered five potential vulnerabilities in the example site:

Password transmitted over HTTP. When a password is transmitted over HTTP, it is vulnerable in many ways. As the password is transmitted over HTTP, network analyser and packet sniffers can easily monitor the traffic and intercept password. HTTP is not considered to be a secure method of user authentication as stated by HTTP specification.

Cookie not marked as HTTPOnly. If a cookie, which is not marked HTTPOnly, sent by the application, is manipulated by the client side code (JavaScript, Java, etc), it could leave the site to XSS vulnerabilities. HTTPOnly cookies cannot be read by client-side script, so marking a cookie as HTTPOnly can provide additional layer of protection against XSS attacks.

Internal server error. The server responded with an HTTP status 500. This

is due to server-side error. Reasons may vary.

Version disclosure. This information can help an attacker gain a greater understanding of the systems in use and potentially develop further attacks targeted at the specific version of application.

Our study shows that Wapiti works like a fuzzer. It detects database injection, cross-site scripting, file handling, etc. The web scanning, fuzzing, crawling, report generation, etc features of OWASP ZAP are very useful for web application testing. Its Web Spider function automatically discovers the hidden links. It allows the tester to understand the structure of the web application. The active scanning feature of OWASP ZAP reports some main vulnerabilities, such as directory browsing, external redirects, session ID in URL rewrite and SQL injection. From all the features that OWASP ZAP offered, fuzzer is the best due to lots of fuzzing plugins that can be used. Also, the process of fuzzing is pretty optimised and fast. The thing that is weird about OWASP ZAP is that, when we scanned web application twice, we got different results. Netsparker community edition offers nice grouping of results and great detail per vulnerability.

Our comparison shows that different tools show different results for the same web application. Wapiti found SQL injection and Blind SQL injection in the application. Several studies show that Wapiti has highest percentage of SQL injection detections. Netsparker does not have Blind SQL Injection module, but is prone to less false positive factors as compared to other tools. OWASP ZAP found cross-site request forgery vulnerability in the web application. OWASP ZAP is prone to high false positive factors as compared to the other tools. It reported SQLi vulnerability as cross-site scripting. So we find that no tool is perfect but each can help to find some basic vulnerabilities. ●

The author is working as a project engineer at CDAC, Mumbai in automation testing. Her research interests are in software engineering, particularly in software testing and reliability, and software metrics